

Asymptotic Analysis

Last Updated : 23 Jul, 2025



Given two algorithms for a task, how do we find out which one is better?

One naive way of doing this is - to implement both the algorithms and run the two programs on your computer for different inputs and see which one takes less time. There are many problems with this approach for the analysis of algorithms.

- It might be possible that for some **inputs**, the first algorithm performs better than the second. And for some **inputs** second performs better.
- It might also be possible that for some inputs, the first algorithm performs better on one **machine**, and the second works better on another **machine** for some other inputs.

*Asymptotic Analysis is the big idea that handles the above issues in analyzing algorithms. In Asymptotic Analysis, we evaluate the performance of an algorithm in terms of input size (we don't measure the actual running time). We calculate, **order of growth** of time taken (or space) by an algorithm in terms of input size. For example linear search grows linearly and Binary Search grows logarithmically in terms of input size.*

For example, let us consider the search problem (searching a given item) in a sorted array.

The solution to above search problem includes:

- [Linear Search](#) (order of growth is linear)
- [Binary Search](#) (order of growth is logarithmic).

To understand how Asymptotic Analysis solves the problems mentioned above in analyzing algorithms,

- let us say:
 - We run the Linear Search on computer A and
 - Binary Search on computer B and
- For small values of input array size n , computer A may take less time.

- But, after a certain value of input array size, the Binary Search will definitely start taking less time compared to the Linear Search even though the Binary Search is being run on a slow machine. Why? After certain value, the machine specific factors would not matter as the value of input would become large.
- The reason is the order of growth of Binary Search with respect to input size is logarithmic while the order of growth of Linear Search is linear.
- **So the machine-dependent constants can always be ignored after a certain value of input size.**
- Let's say the constant for machine A is 0.2 and the constant for B is 1000 which means that A is 5000 times more powerful than B.

Input Size	Running time on A	Running time on B
10	2 sec	~ 1 h
100	20 sec	~ 1.8 h
10^6	~ 55.5 h	~ 5.5 h
10^9	~ 6.3 years	~ 8.3 h

Running times for this example:

- Linear Search running time in seconds on A: $0.2 * n$
- Binary Search running time in seconds on B: $1000 * \log(n)$

Does Asymptotic Analysis always work?

Asymptotic Analysis is not perfect, but that's the best way available for analyzing algorithms. For example, say there are two sorting algorithms that take $1000n \log n$ and $2n \log n$ time respectively on a machine. Both of these algorithms are asymptotically the same (order of growth is $n \log n$). So, With Asymptotic Analysis, we can't judge which one is better as we ignore constants in Asymptotic Analysis. For example, asymptotically [Heap Sort](#) is better than [Quick Sort](#), but Quick Sort takes less time in practice.

Also, in Asymptotic analysis, we always talk about input sizes larger than a constant value. It might be possible that those large inputs are never given to your software and an asymptotically slower algorithm always performs better for your particular situation. So, you may end up choosing an algorithm that is Asymptotically slower but faster for your software.

Order of Growth

Visit Course ↗

Comment

More info ▼

Advertise with us

Explore

DSA Fundamentals ▼

Data Structures ▼

Algorithms ▼

Advanced ▼

Interview Preparation ▼

Practice Problem ▼

Do Not Sell or Share My Personal Information